



city-zen

Η πόλη στα χέρια σου.

City-Zen

<<Η πόλη στα χέρια σου.>>

Project-code-v01

ΜΕΛΗ ΟΜΑΔΑΣ

ΑΥΓΟΥΣΤΙΝΟΥ ΑΓΑΠΗ 1093327

ΔΟΥΒΡΗ ΑΓΓΕΛΙΚΗ 1097441

ΚΑΤΣΑΝΤΑΣ ΘΕΟΔΩΡΟΣ 1097459

ΜΟΝΑΣΤΗΡΙΩΤΗΣ ΗΛΙΑΣ 1093436

ΣΚΟΥΤΑΣ ΚΩΝΣΤΑΝΤΙΝΟΣ 1093498

ΣΥΝΤΑΚΤΕΣ USE-CASE.v02

Editors

ΑΥΓΟΥΣΤΙΝΟΥ ΑΓΑΠΗ 1093327

Πίνακες:

Διαχείριση Χρηστών:

user: βασικός πίνακας για credentials/πληροφορίες σύνδεσης (username, password_hash, email, τηλέφωνο).

-- table for users

```
CREATE TABLE user (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(100) NOT NULL UNIQUE,  
    password_hash VARCHAR(255) NOT NULL,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    telephone_number VARCHAR(20) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

citizen και municipality: εξειδικεύουν τον user σε δύο ρόλους: πολίτης και δήμος, με ξένο κλειδί στο user (το user_id).

-- table for municipality

```
CREATE TABLE municipality (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT NOT NULL UNIQUE,  
    municipality VARCHAR(100) NOT NULL,  
    contact_email VARCHAR(100),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (user_id) REFERENCES user(id) ON DELETE CASCADE  
);
```

-- table for citizens

```
CREATE TABLE citizen (  

```

```

id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL UNIQUE,
municipality_id INT NOT NULL,
name VARCHAR(100),
points INT DEFAULT 0,
FOREIGN KEY (user_id) REFERENCES user(id) ON DELETE CASCADE,
FOREIGN KEY (municipality_id) REFERENCES municipality(id) ON DELETE
CASCADE
);

```

Διαχείριση Αναφορών:

report: ο κύριος πίνακας αναφοράς, με χρήσιμα πεδία (status, priority, category, etc.).

location: σωστά διαχωρισμένος για επαναχρησιμοποίηση τοποθεσιών.

report_media: media που σχετίζονται με αναφορές.

-- table for reports

```

CREATE TABLE report (
  id INT AUTO_INCREMENT PRIMARY KEY,
  citizen_id INT NOT NULL,
  location_id INT NOT NULL,
  category ENUM('Lighting', 'Roadworks', 'Other') NOT NULL DEFAULT 'Other',
  description TEXT NOT NULL,
  priority ENUM('High', 'Medium', 'Low') NOT NULL DEFAULT 'Medium',
  status ENUM('Pending', 'Rejected', 'In Progress', 'Completed') NOT NULL DEFAULT
'Pending',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (citizen_id) REFERENCES citizen(id),
  FOREIGN KEY (location_id) REFERENCES location(id)
);

```

-- table for location

```
CREATE TABLE location (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    latitude DOUBLE NOT NULL,  
    longitude DOUBLE NOT NULL,  
    address VARCHAR(255) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- table for report media

```
CREATE TABLE report_media (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    report_id INT NOT NULL,  
    media_url VARCHAR(500) NOT NULL,  
    media_type ENUM('image', 'video') DEFAULT 'image',  
    uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (report_id) REFERENCES report(id) ON DELETE CASCADE  
);
```

Σχόλια και Επικοινωνία:

citizen_comment: σωστή δομή με parent_comment_id για υποστήριξη απαντήσεων (nested threads).

municipality_comment: ξεχωριστός πίνακας για σχόλια από τον δήμο — ενισχύει τη διαφάνεια.

-- table for citizen comments

```
CREATE TABLE citizen_comment (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    report_id INT NOT NULL,
```

```

    citizen_id INT NOT NULL,
    parent_comment_id INT NULL,
    comment TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (report_id) REFERENCES report(id),
    FOREIGN KEY (citizen_id) REFERENCES citizen(id),
    FOREIGN KEY (parent_comment_id) REFERENCES citizen_comment(id) ON
DELETE CASCADE
);

```

-- table for municipality comments

```

CREATE TABLE municipality_comment (
    id INT AUTO_INCREMENT PRIMARY KEY,
    municipality_id INT NOT NULL,
    report_id INT NOT NULL,
    comment TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (municipality_id) REFERENCES municipality(id) ON DELETE
CASCADE,
    FOREIGN KEY (report_id) REFERENCES report(id) ON DELETE CASCADE
);

```

AI & Bot Συνομιλίες:

ai_conversation/ ai_chat: καλός διαχωρισμός συνομιλίας και επιμέρους μηνυμάτων.

Περιλαμβάνει χρήσιμες πληροφορίες (π.χ. action_type, report_id) — έτοιμο για analytics/κατηγοριοποίηση.

-- table for ai conversation

```

CREATE TABLE ai_conversation (
    id INT AUTO_INCREMENT PRIMARY KEY,

```

```

    citizen_id INT NOT NULL,
    started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ended_at TIMESTAMP,
    resolved BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (citizen_id) REFERENCES citizen(id)
);

-- table for ai chat
CREATE TABLE ai_chat (
    id INT AUTO_INCREMENT PRIMARY KEY,
    conversation_id INT NOT NULL,
    citizen_id INT NOT NULL,
    message TEXT NOT NULL,
    is_user BOOLEAN NOT NULL,
    report_id INT,
    action_type ENUM('submit_report', 'check_status', 'get_info', 'feedback', 'other'),
    feedback_rating TINYINT,
    user_feedback TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (conversation_id) REFERENCES ai_conversation(id) ON DELETE CASCADE,
    FOREIGN KEY (citizen_id) REFERENCES citizen(id),
    FOREIGN KEY (report_id) REFERENCES report(id)
);

```

Ειδοποιήσεις & Ανακοινώσεις:

notification: πλήρης πίνακας με αναφορά στο action_type, citizen_id, report_id.

municipality_post: ανακοινώσεις από δήμο, με δυνατότητα εικόνας/media.

```

-- table for notifications

```

```
CREATE TABLE notification (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    citizen_id INT NOT NULL,  
    report_id INT DEFAULT NULL,  
    message TEXT NOT NULL,  
    is_read BOOLEAN DEFAULT FALSE,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    action_type ENUM(  
        'report_submitted',  
        'status_updated',  
        'points_awarded',  
        'points_deducted',  
        'admin_announcement',  
        'general'  
    ) NOT NULL,  
    link_url VARCHAR(255),  
    FOREIGN KEY (citizen_id) REFERENCES citizen(id),  
    FOREIGN KEY (report_id) REFERENCES report(id)  
);
```

-- table for municipality announcements

```
CREATE TABLE municipality_post (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    municipality_id INT NOT NULL,  
    title VARCHAR(255) NOT NULL,  
    text TEXT NOT NULL,  
    media_url VARCHAR(500),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```



```
FOREIGN KEY (municipality_id) REFERENCES municipality(id)
);
```

Πόντοι, Επιβραβεύσεις και Κυρώσεις:

points / points_redemption / false_report / penalty: ολοκληρωμένο point system με tracking αιτίας (reason), επιβραβεύσεις και ποινές.

-- table for points

```
CREATE TABLE points (
    id INT AUTO_INCREMENT PRIMARY KEY,
    citizen_id INT NOT NULL,
    report_id INT DEFAULT NULL,
    points_awarded INT NOT NULL,
    reason VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (citizen_id) REFERENCES citizen(id),
    FOREIGN KEY (report_id) REFERENCES report(id)
);
```

-- table for point redemptions

```
CREATE TABLE points_redemption (
    id INT AUTO_INCREMENT PRIMARY KEY,
    citizen_id INT NOT NULL,
    points_spent INT NOT NULL,
    reward_description VARCHAR(255) NOT NULL,
    redeemed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (citizen_id) REFERENCES citizen(id)
);
```

-- table for false reports

```
CREATE TABLE false_report (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    report_id INT NOT NULL UNIQUE,  
    detected_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    reason TEXT,  
    FOREIGN KEY (report_id) REFERENCES report(id) ON DELETE CASCADE  
);
```

-- table for penalties

```
CREATE TABLE penalty (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    citizen_id INT NOT NULL,  
    false_report_id INT NOT NULL,  
    points_deducted INT NOT NULL,  
    reason TEXT,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (citizen_id) REFERENCES citizen(id),  
    FOREIGN KEY (false_report_id) REFERENCES report(id)  
);
```

Αξιολόγηση Αναφορών:

report_rating: Δυνατότητα αξιολόγησης επίλυσης από τον πολίτη με επιμέρους βαθμολογίες (χρόνος, επικοινωνία, ποιότητα). Ενισχύει τη διαφάνεια και ανατροφοδότηση προς τον δήμο.

-- table for report rating

```
CREATE TABLE report_rating (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    report_id INT NOT NULL UNIQUE,
```

```

    citizen_id INT NOT NULL,

    resolution_time_rating TINYINT NOT NULL CHECK (resolution_time_rating
BETWEEN 1 AND 5),

    communication_rating TINYINT NOT NULL CHECK (communication_rating
BETWEEN 1 AND 5),

    solution_quality_rating TINYINT NOT NULL CHECK (solution_quality_rating
BETWEEN 1 AND 5),

    comment TEXT,

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (report_id) REFERENCES report(id),

    FOREIGN KEY (citizen_id) REFERENCES citizen(id)

);

```

Αιτήματα Πολιτών:

request: Καταγράφει αιτήματα πολιτών προς τον δήμο με κατάσταση επεξεργασίας (Pending, Approved, Rejected).

-- table for requests

```

CREATE TABLE request (

    id INT AUTO_INCREMENT PRIMARY KEY,

    citizen_id INT NOT NULL,

    municipality_id INT NOT NULL,

    description TEXT NOT NULL,

    status ENUM('Pending', 'Approved', 'Rejected') NOT NULL,

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

    FOREIGN KEY (citizen_id) REFERENCES citizen(id),

    FOREIGN KEY (municipality_id) REFERENCES municipality(id)

);

```

Procedures:

SubmitReport: Εισάγει νέα αναφορά από πολίτη με τις λεπτομέρειες της και δημιουργεί σχετική ειδοποίηση στον πολίτη.

UpdateReportStatus: Ενημερώνει την κατάσταση μιας αναφοράς και απονέμει ή αφαιρεί πόντους στον πολίτη, με ειδοποίηση για την αλλαγή κατάστασης.

AwardPoints: Απονέμει πόντους στον πολίτη για την ολοκλήρωση μιας αναφοράς και στέλνει ειδοποίηση.

DeductPoints: Αφαιρεί πόντους από τον πολίτη για λάθος αναφορά και αποστέλλει ειδοποίηση για την ποινή.

SubmitRequest: Υποβάλλει αίτημα από πολίτη στον δήμο και δημιουργεί σχετική ειδοποίηση για την υποβολή του.

PublishMunicipalityPost: Δημιουργεί ανάρτηση από τον δήμο και ενημερώνει όλους τους πολίτες.

GetOtherCitizensInProgressReports: Εμφανίζει αναφορές σε εκκρεμότητα από άλλους πολίτες στο ίδιο δήμο.

AddCitizenComment: Επιτρέπει σε πολίτη να σχολιάσει μια αναφορά άλλου πολίτη και ενημερώνει τον κάτοχο της αναφοράς.

SubmitFeedback: Υποβάλλει ανατροφοδότηση από τον πολίτη μέσω της AI συνομιλίας και καταγράφει τη βαθμολογία και το σχόλιο.

CreateCitizen: Δημιουργεί λογαριασμό πολίτη με τα προσωπικά του στοιχεία.

CreateMunicipality: Δημιουργεί λογαριασμό δήμου με τα στοιχεία επαφής και σύνδεσης.

GetCitizenReports: Εμφανίζει τις αναφορές που έχει υποβάλει ο συγκεκριμένος πολίτης.

GetReportsForMunicipality: Εμφανίζει τις αναφορές των πολιτών για ένα συγκεκριμένο δήμο.

AddMunicipalityComment: Επιτρέπει στον δήμο να σχολιάσει μια αναφορά και ειδοποιεί τον πολίτη.

RedeemPoints: Επιτρέπει στον πολίτη να εξαργυρώσει πόντους για ανταμοιβή και καταγράφει την εξόφληση.

GetCitizenPoints: Εμφανίζει τους πόντους που διαθέτει ο πολίτης.

GetCitizenNotifications: Εμφανίζει όλες τις ειδοποιήσεις για έναν πολίτη.

EndAIConversation: Τερματίζει τη συνομιλία AI αν το πρόβλημα έχει λυθεί.

auto_close_inactive_conversations: Κλείνει αυτόματα τις ανενεργές συνομιλίες AI μετά από 1 ώρα αδράνειας.

EditCitizenFullProfileWithPassword: Επιτρέπει στον πολίτη να επεξεργαστεί το προφίλ του, περιλαμβανομένων των στοιχείων σύνδεσης.

ReplyToCitizenComment: Επιτρέπει σε πολίτη να απαντήσει σε σχόλιο άλλου πολίτη σε αναφορά και ενημερώνει τον αρχικό σχολιαστή.

-- citizen submits report and gets notification

DELIMITER //

CREATE PROCEDURE SubmitReport (

IN p_citizen_id INT,

IN p_latitude DOUBLE,

IN p_longitude DOUBLE,

IN p_address VARCHAR(255),

IN p_category ENUM('lighting', 'roadworks', 'other'),

IN p_description TEXT,

IN p_priority ENUM('High', 'Medium', 'Low'),

IN p_media_url VARCHAR(500),

IN p_media_type ENUM('image', 'video')

)

BEGIN

DECLARE loc_id INT;

DECLARE report_id INT;

INSERT INTO location (latitude, longitude, address)

VALUES (p_latitude, p_longitude, p_address);

SET loc_id = LAST_INSERT_ID();

INSERT INTO report (citizen_id, location_id, category, description, priority)

```
VALUES (p_citizen_id, loc_id, p_category, p_description, p_priority);
```

```
SET report_id = LAST_INSERT_ID();
```

```
IF p_media_url IS NOT NULL AND p_media_url != " THEN
```

```
    INSERT INTO report_media (report_id, media_url, media_type)
```

```
    VALUES (report_id, p_media_url, p_media_type);
```

```
END IF;
```

```
INSERT INTO notification (citizen_id, message, action_type, report_id)
```

```
VALUES (
```

```
    p_citizen_id,
```

```
    CONCAT('Your report has been submitted successfully. Report ID: ', report_id),
```

```
    'report_submitted',
```

```
    report_id
```

```
);
```

```
END //
```

```
DELIMITER ;
```

```
-- municipality changes status, citizen gets notified and if new status is 'in progress'  
gets awarded points, if it is 'rejected' gets deducted points
```

```
DELIMITER //
```

```
CREATE PROCEDURE UpdateReportStatus (
```

```
    IN p_report_id INT,
```

```

    IN p_citizen_id INT,
    IN p_new_status ENUM('Pending', 'Rejected', 'In Progress', 'Completed')
)
BEGIN
    UPDATE report
    SET status = p_new_status
    WHERE id = p_report_id;

    IF p_new_status = 'In progress' THEN
        CALL AwardPoints(p_citizen_id, p_report_id, 10, 'Report completed');
    END IF;

    IF p_new_status = 'Rejected' THEN
        CALL DeductPoints(p_citizen_id, p_report_id, 10, 'Report was rejected');
    END IF;

    INSERT INTO notification (citizen_id, message, action_type, report_id)
    VALUES (
        p_citizen_id,
        CONCAT('Status of your report (ID: ', p_report_id, ') has been updated to ',
        p_new_status),
        'status_updated',
        p_report_id
    );
END //

DELIMITER ;

```

```

-- citizen gets awarded points and gets notification
DELIMITER //

CREATE PROCEDURE AwardPoints (
    IN p_citizen_id INT,
    IN p_report_id INT,
    IN p_points INT,
    IN p_reason VARCHAR(255)
)
BEGIN
    INSERT INTO points (citizen_id, report_id, points_awarded, reason)
    VALUES (p_citizen_id, p_report_id, p_points, p_reason);

    UPDATE citizen
    SET points = points + p_points
    WHERE id = p_citizen_id;

    INSERT INTO notification (citizen_id, message, action_type, report_id)
    VALUES (
        p_citizen_id,
        CONCAT('You have been awarded ', p_points, ' points: ', p_reason),
        'points_awarded',
        p_report_id
    );
END //

```


DELIMITER ;

-- citizen gets deducted points and gets notification

DELIMITER //

CREATE PROCEDURE DeductPoints(

IN p_citizen_id INT,

IN p_false_report_id INT,

IN p_points_deducted INT,

IN p_reason TEXT

)

BEGIN

INSERT INTO penalty (citizen_id, false_report_id, points_deducted, reason)

VALUES (p_citizen_id, p_false_report_id, p_points_deducted, p_reason);

UPDATE citizen

SET points = points - p_points_deducted

WHERE id = p_citizen_id;

INSERT INTO notification (citizen_id, message, action_type, report_id)

VALUES (

p_citizen_id,

CONCAT('Warning: ', p_points_deducted, ' points deducted. Reason: ', p_reason),

'points_deducted',

p_false_report_id

);

```
END //
```

```
DELIMITER ;
```

```
-- citizen submits a request
```

```
DELIMITER //
```

```
CREATE PROCEDURE SubmitRequest (
```

```
    IN p_citizen_id INT,
```

```
    IN p_municipality_id INT,
```

```
    IN p_description TEXT
```

```
)
```

```
BEGIN
```

```
    DECLARE new_request_id INT;
```

```
    INSERT INTO request (citizen_id, municipality_id, description, status)
```

```
    VALUES (p_citizen_id, p_municipality_id, p_description, 'Pending');
```

```
    SET new_request_id = LAST_INSERT_ID();
```

```
    INSERT INTO notification (citizen_id, message, action_type, link_url)
```

```
    VALUES (
```

```
        p_citizen_id,
```

```
        CONCAT('Your request has been submitted successfully. Request ID: ',  
new_request_id),
```

```
        'general',
```

```
        CONCAT('/requests/', new_request_id)
```

```
    );
```

END //

DELIMITER ;

DELIMITER //

CREATE PROCEDURE PublishMunicipalityPost (

IN p_municipality_id INT,

IN p_title VARCHAR(255),

IN p_content TEXT,

IN p_media_url VARCHAR(500)

)

BEGIN

DECLARE post_id INT;

INSERT INTO municipality_post (municipality_id, title, text, media_url)

VALUES (p_municipality_id, p_title, p_content, p_media_url);

SET post_id = LAST_INSERT_ID();

INSERT INTO notification (citizen_id, message, action_type)

SELECT id, CONCAT('New municipality announcement: ', p_title),
'admin_announcement'

FROM citizen;

END //

DELIMITER ;

-- citizen views other citizens' reports that are in progress

DELIMITER //

CREATE PROCEDURE GetOtherCitizensInProgressReports (

 IN p_citizen_id INT

)

BEGIN

 DECLARE v_municipality_id INT;

 SELECT municipality_id INTO v_municipality_id

 FROM citizen

 WHERE id = p_citizen_id;

 SELECT

 r.id AS report_id,

 r.description,

 r.status,

 r.created_at,

 l.address AS location_address,

 l.latitude,

 l.longitude,

 c.id AS reporter_id,

 c.name AS reporter_name,

 GROUP_CONCAT(DISTINCT rm.media_url SEPARATOR ', ') AS media_urls,

 GROUP_CONCAT(DISTINCT CONCAT(cc.comment, ' (by Citizen ID: ', cc.citizen_id, ' ') SEPARATOR ' || ') AS comments

```

FROM report r
JOIN citizen c ON r.citizen_id = c.id
JOIN location l ON r.location_id = l.id
LEFT JOIN report_media rm ON r.id = rm.report_id
LEFT JOIN citizen_comment cc ON r.id = cc.report_id
WHERE r.status = 'In Progress'
      AND c.municipality_id = v_municipality_id
      AND c.id != p_citizen_id
GROUP BY r.id
ORDER BY r.created_at ASC;
END //

```

```

DELIMITER ;

```

```

-- citizen comments on another citizen's report and citizen gets notification

```

```

DELIMITER //

```

```

CREATE PROCEDURE AddCitizenComment (
    IN p_citizen_id INT,
    IN p_report_id INT,
    IN p_comment TEXT
)
BEGIN
    DECLARE report_owner_id INT;

    INSERT INTO citizen_comment (report_id, citizen_id, comment)

```

```
VALUES (p_report_id, p_citizen_id, p_comment);
```

```
SELECT citizen_id INTO report_owner_id
```

```
FROM report
```

```
WHERE id = p_report_id;
```

```
IF report_owner_id IS NOT NULL AND report_owner_id != p_citizen_id THEN
```

```
    INSERT INTO notification (citizen_id, message, action_type, report_id)
```

```
    VALUES (
```

```
        report_owner_id,
```

```
        CONCAT('Another citizen commented on your report (ID: ', p_report_id, ').'),
```

```
        'general',
```

```
        p_report_id
```

```
    );
```

```
END IF;
```

```
END //
```

```
DELIMITER ;
```

```
-- citizen submits feedback in ai chat
```

```
DELIMITER //
```

```
CREATE PROCEDURE SubmitFeedback (
```

```
    IN p_citizen_id INT,
```

```
    IN p_report_id INT,
```

```
    IN p_rating TINYINT,
```

```

        IN p_comment TEXT
    )
BEGIN
    DECLARE conv_id INT;

    INSERT INTO ai_conversation (citizen_id)
    VALUES (p_citizen_id);
    SET conv_id = LAST_INSERT_ID();

    INSERT INTO ai_chat (conversation_id, citizen_id, message, is_user, report_id,
action_type, feedback_rating, user_feedback)
    VALUES (conv_id, p_citizen_id, p_comment, TRUE, p_report_id, 'feedback', p_rating,
p_comment);
END //

DELIMITER ;

-- citizen creates account
DELIMITER //

CREATE PROCEDURE CreateCitizen(
    IN p_username VARCHAR(100),
    IN p_password_hash VARCHAR(255),
    IN p_email VARCHAR(100),
    IN p_telephone_number VARCHAR(20),
    IN p_municipality_id INT
)

```

```
BEGIN

    INSERT INTO user (username, password_hash, email, telephone_number)
    VALUES (p_username, p_password_hash, p_email, p_telephone_number);

    INSERT INTO citizen (user_id, municipality_id)
    VALUES (LAST_INSERT_ID(), p_municipality_id);
END //
```

```
DELIMITER ;
```

```
-- municipality creates account
```

```
DELIMITER //
```

```
CREATE PROCEDURE CreateMunicipality(
    IN p_username VARCHAR(100),
    IN p_password_hash VARCHAR(255),
    IN p_email VARCHAR(100),
    IN p_telephone_number VARCHAR(20),
    IN p_municipality VARCHAR(100),
    IN p_contact_email VARCHAR(100)
)
BEGIN

    INSERT INTO user (username, password_hash, email, telephone_number)
    VALUES (p_username, p_password_hash, p_email, p_telephone_number);

    INSERT INTO municipality (user_id, municipality, contact_email)
```



```
VALUES (LAST_INSERT_ID(), p_municipality, p_contact_email);  
END //
```

```
DELIMITER ;
```

```
-- citizen views their own reports
```

```
DELIMITER //
```

```
CREATE PROCEDURE GetCitizenReports(  
    IN p_citizen_id INT  
)  
BEGIN
```

```
    SELECT
```

```
        r.id AS report_id,
```

```
        r.description,
```

```
        r.category,
```

```
        r.priority,
```

```
        r.status,
```

```
        r.created_at,
```

```
        l.address AS location_address,
```

```
        l.latitude,
```

```
        l.longitude,
```

```
    (  
        SELECT GROUP_CONCAT(rm.media_url SEPARATOR ', ')
```

```
        FROM report_media rm
```

```
        WHERE rm.report_id = r.id
```

```

    ) AS media_urls,
    (
        SELECT GROUP_CONCAT(CONCAT(cit.name, ': ', cc.comment) SEPARATOR ' || ')
        FROM citizen_comment cc
        JOIN citizen cit ON cit.id = cc.citizen_id
        WHERE cc.report_id = r.id
    ) AS comments
FROM report r
JOIN location l ON r.location_id = l.id
WHERE r.citizen_id = p_citizen_id
ORDER BY r.created_at DESC;
END //

```

```

DELIMITER ;

```

```

-- municipality views citizens' reports

```

```

DELIMITER //

```

```

CREATE PROCEDURE GetReportsForMunicipality(
    IN p_municipality_id INT
)
BEGIN
    SELECT
        r.id AS report_id,
        r.description,
        r.category,

```

```

r.priority,
r.status,
r.created_at,
l.address AS location_address,
l.latitude,
l.longitude,
(
    SELECT GROUP_CONCAT(rm.media_url SEPARATOR ', ')
    FROM report_media rm
    WHERE rm.report_id = r.id
) AS media_urls,
(
    SELECT GROUP_CONCAT(CONCAT(cit.name, ': ', cc.comment) SEPARATOR ' || ')
    FROM citizen_comment cc
    JOIN citizen cit ON cit.id = cc.citizen_id
    WHERE cc.report_id = r.id
) AS comments
FROM report r
INNER JOIN location l ON r.location_id = l.id
INNER JOIN citizen c ON r.citizen_id = c.id
WHERE c.municipality_id = p_municipality_id
ORDER BY r.created_at DESC;
END //

DELIMITER ;

```

-- municipality comments on a report and citizen gets notified

DELIMITER //

CREATE PROCEDURE AddMunicipalityComment(

IN p_municipality_id INT,

IN p_report_id INT,

IN p_comment TEXT

)

BEGIN

INSERT INTO municipality_comment (municipality_id, report_id, comment)

VALUES (p_municipality_id, p_report_id, p_comment);

INSERT INTO notification (citizen_id, report_id, message, action_type)

SELECT r.citizen_id, r.id, CONCAT('Υπάρχει νέο σχόλιο από τον Δήμο στη αναφορά σας. '), 'general'

FROM report r

WHERE r.id = p_report_id;

END //

DELIMITER ;

-- citizen redeems points

DELIMITER //

CREATE PROCEDURE RedeemPoints(

IN p_citizen_id INT,

```

    IN p_points_spent INT,
    IN p_reward_description VARCHAR(255)
)
BEGIN
    DECLARE current_points INT;

    SELECT points INTO current_points
    FROM citizen
    WHERE id = p_citizen_id;

    IF current_points >= p_points_spent THEN

        UPDATE citizen
        SET points = points - p_points_spent
        WHERE id = p_citizen_id;

        INSERT INTO points_redemption (citizen_id, points_spent, reward_description)
        VALUES (p_citizen_id, p_points_spent, p_reward_description);
    ELSE
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Not enough points';
    END IF;
END //

DELIMITER ;

```

-- citizen views how many points they have

DELIMITER //

CREATE PROCEDURE GetCitizenPoints(

IN p_citizen_id INT

)

BEGIN

SELECT points

FROM citizen

WHERE id = p_citizen_id;

END //

DELIMITER ;

-- citizen views all notifications

DELIMITER //

-- citizen views all notifications

CREATE PROCEDURE GetCitizenNotifications(

IN p_citizen_id INT

)

BEGIN

SELECT * FROM notification

WHERE citizen_id = p_citizen_id

ORDER BY created_at DESC;

END //

DELIMITER ;

-- ai conversation is ended if the issue is resolved

DELIMITER //

CREATE PROCEDURE EndAIConversation (

IN p_conversation_id INT

)

BEGIN

UPDATE ai_conversation

SET ended_at = CURRENT_TIMESTAMP, resolved = TRUE

WHERE id = p_conversation_id;

END //

DELIMITER ;

-- ai conversation is ended if user is inactive for more than one hour

DELIMITER //

CREATE EVENT auto_close_inactive_conversations

ON SCHEDULE EVERY 5 MINUTE

DO

```

BEGIN
    UPDATE ai_conversation ac
    SET ac.ended_at = CURRENT_TIMESTAMP, ac.resolved = FALSE
    WHERE ac.ended_at IS NULL
    AND (
        SELECT MAX(chat.created_at)
        FROM ai_chat chat
        WHERE chat.conversation_id = ac.id
    ) < NOW() - INTERVAL 1 HOUR;
END //

```

```

DELIMITER ;

```

```

-- citizen edits profile

```

```

DELIMITER //

```

```

CREATE PROCEDURE EditCitizenFullProfileWithPassword (

```

```

    IN p_citizen_id INT,
    IN p_new_name VARCHAR(100),
    IN p_new_municipality_id INT,
    IN p_new_username VARCHAR(100),
    IN p_new_email VARCHAR(100),
    IN p_new_telephone_number VARCHAR(20),
    IN p_new_password_hash VARCHAR(255)

```

```

)

```

```

BEGIN

```



```
DECLARE v_user_id INT;
```

```
SELECT user_id INTO v_user_id
```

```
FROM citizen
```

```
WHERE id = p_citizen_id;
```

```
UPDATE citizen
```

```
SET
```

```
    name = p_new_name,
```

```
    municipality_id = p_new_municipality_id
```

```
WHERE id = p_citizen_id;
```

```
UPDATE user
```

```
SET
```

```
    username = p_new_username,
```

```
    email = p_new_email,
```

```
    telephone_number = p_new_telephone_number,
```

```
    password_hash = p_new_password_hash
```

```
WHERE id = v_user_id;
```

```
END //
```

```
DELIMITER ;
```

```
-- citizen replies to a citizen that has commented under their report
```

```
DELIMITER //
```

```

CREATE PROCEDURE ReplyToCitizenComment (
    IN p_citizen_id INT,
    IN p_report_id INT,
    IN p_parent_comment_id INT,
    IN p_reply_text TEXT
)
BEGIN
    DECLARE original_commenter_id INT;

    INSERT INTO citizen_comment (report_id, citizen_id, comment,
parent_comment_id)
VALUES (p_report_id, p_citizen_id, p_reply_text, p_parent_comment_id);

    SELECT citizen_id INTO original_commenter_id
    FROM citizen_comment
    WHERE id = p_parent_comment_id;

    IF original_commenter_id IS NOT NULL AND original_commenter_id != p_citizen_id
    THEN
        INSERT INTO notification (citizen_id, message, action_type, report_id)
        VALUES (
            original_commenter_id,
            CONCAT('A citizen replied to your comment on report ID ', p_report_id),
            'general',
            p_report_id
        );
    END IF;
END //

```

DELIMITER